

INSPECTAIR

Coding Rules

Programmierrichtlinien und Konventionen

Projekt: InspectAir – Luftqualitätsmessgerät
Version: 2.0
Datum: Februar 2026
Autoren: Maximilian Liebl, Cedric Stroh, Jannis Messer
Modul: Mikrocomputertechnik 1, DHBW Ravensburg
Dozent: Thomas Stingl (Airbus)

Dieses Dokument wurde unter Verwendung von KI-Tools (Claude, Anthropic) zur Überprüfung erstellt.

Inhaltsverzeichnis

1	Programmiersprache und Umgebung	2
2	Formatierung	2
3	Namenskonventionen	2
4	Dokumentation	2
5	Fehlerbehandlung	3
6	Verbotene Praktiken	3
7	Versionskontrolle	4
8	Build-Konfiguration	4
9	Änderungshistorie	4

1 Programmiersprache und Umgebung

Parameter	Wert
Sprache	C++ (Arduino Framework)
IDE	VSCode mit PlatformIO
Board	esp32-s3-devkitc-1
Framework	Arduino (Espressif32)
Flash Mode	QIO 80 MHz, OPI PSRAM
Upload Speed	921600 Baud
Monitor Speed	115200 Baud

2 Formatierung

Regel	Beschreibung
Einrückung	4 Leerzeichen (keine Tabs)
Zeilenlänge	Maximal 100 Zeichen
Encoding	UTF-8
Zeilenende	LF (Unix-Style)
Klammern	Öffnende geschweifte Klammer auf gleicher Zeile (K&R-Stil)
Leerzeilen	Eine Leerzeile zwischen logischen Blöcken und Funktionen

3 Namenskonventionen

Element	Konvention	Beispiel
Variablen	snake_case	sensor_data, last_read_time
Funktionen	snake_case	read_sensors(), ui_init()
Konstanten / Defines	UPPER_SNAKE_CASE	PRESENCE_TIMEOUT_DIM_MS
Structs / Enums	PascalCase	SensorReadings, DisplayState
Enum-Werte	UPPER_SNAKE_CASE	UI_SCREEN_OVERVIEW, STATE_ACTIVE
Klassen	PascalCase	PowerManager, WifiClock
Private Member	Präfix _	_state, _fadeCurrent
Typedefs	Suffix _Data	AHT20_Data, PMS5003_Data
Header Guards	UPPER_SNAKE_CASE mit _H	POWER_MANAGER_H
Dateinamen	snake_case.cpp/.h	power_manager.cpp, sensor_filter.h
Pin-Defines	Präfix nach Funktion	TFT_MOSI, I2C_SDA, CO2_RX

4 Dokumentation

Alle öffentlichen Funktionen, Klassen und Strukturen werden mit Doxygen-Kommentaren versehen:

```
/**
 * @brief Aktualisiert alle Sensorwerte auf dem Display
 * @param readings Aktuelle Sensor-Messwerte
 */
void ui_updateSensorValues(const SensorReadings& readings);
```

Inline-Kommentare erklären nicht-offensichtliche Logik:

```
// 220uF Stuetzkondensator puffert Stromspitzen des MH-Z19C
delay(100); // Warten bis Sensor-Versorgung stabil
```

5 Fehlerbehandlung

Da Exceptions auf ESP32 nicht verfügbar sind, wird folgendes Schema verwendet:

- Jeder Sensor-Treiber gibt ein `ok`-Flag in seiner Datenstruktur zurück (z. B. `MHZ19C_Data.valid`).
- Fehlerzustände werden über `Serial.println()` auf der Debug-Konsole protokolliert.
- Die `main.cpp` prüft bei jedem Lesezyklus den Rückgabewert und zeigt fehlerhafte Sensoren auf dem Display an.
- Ein Watchdog-Timer (WDT) überwacht die Hauptschleife und löst bei Blockierung einen Reset aus.

Beispiel aus dem Sensor-Treiber:

```
MHZ19C_Data mhz19c_read() {
    MHZ19C_Data data;
    data.co2_ppm = myMHZ19.getC02();
    data.valid = (data.co2_ppm > 0 && data.co2_ppm < 5000);
    return data;
}
```

6 Verbotene Praktiken

Verboten	Begründung
<code>delay()</code> in <code>loop()</code>	Blockiert LVGL-Rendering und Sensor-Abfragen. Stattdessen <code>millis()</code> -basiertes Timing verwenden.
<code>String</code> Klasse	Heap-Fragmentierung auf ESP32. Stattdessen <code>char[]</code> , <code>snprintf()</code> oder <code>lv_label_set_text_fmt()</code> verwenden.
Globale Variablen	Nur <code>static</code> -Variablen in <code>main.cpp</code> und <code>extern</code> -Singleton-Instanzen (z. B. <code>powerManager</code>) erlaubt.
Magic Numbers	Alle Konstanten als <code>#define</code> oder <code>static const</code> mit sprechendem Namen in <code>pins.h</code> , <code>colors.h</code> oder <code>app_config.h</code> .
Dynamische Allokation in <code>loop()</code>	Kein <code>new/malloc</code> in der Hauptschleife. LVGL-Objekte werden einmalig in <code>ui_init()</code> erstellt.
Blocking I/O	Sensor-Reads mit Timeout absichern; niemals endlos auf Daten warten.

7 Versionskontrolle

Regel	Beschreibung
Repository	GitHub (privat), Git-basiert
Branching	<code>main</code> (stabil) + Feature-Branches (<code>feature/sensor-xyz</code>)
Commit-Format	Conventional Commits: <code>feat:</code> , <code>fix:</code> , <code>docs:</code> , <code>refactor:</code>
Merge-Strategie	Feature-Branch → Pull Request → Review → Merge in <code>main</code>
<code>.gitignore</code>	Build-Artefakte (<code>.pio/</code>), IDE-Konfiguration (<code>.vscode/</code>), Credentials

8 Build-Konfiguration

Zentrale Build-Flags in `platformio.ini`:

```
build_flags =
  -DARDUINO_USB_CDC_ON_BOOT=1
  -DBOARD_HAS_PSRAM
  -DLV_CONF_INCLUDE_SIMPLE
  -DLV_COLOR_DEPTH=16
  -DLV_COLOR_16_SWAP=1
  -DLV_MEM_SIZE="(96U * 1024U)"
  -DLV_FONT_MONTSERRAT_16=1
```

Pin-Definitionen sind zentral in `pins.h` gesammelt und dürfen nicht in einzelnen Modulen hart codiert werden.

9 Änderungshistorie

Version	Datum	Autor	Änderung
1.0	Januar 2026	Team	Erstversion
2.0	Februar 2026	Team	Ergänzung: Watchdog, LVGL-spezifische Regeln, Build-Flags, Private-Member-Konvention, Dateinamen-Konvention.